AMRITA VISHWA VIDYAPEETHAM | School of Engineering Chennai Campus

Department of Computer Science and Engineering
2024 CSE BATCH
23CSE463 – Quantum Computing

KANTAMRAJU SREEJA

CH.SC.U4CSE24255

WORKSHEET-5

WORKSHEET- 5

EX.NO : 5

DATE : 10-10-26

TITLE : Design and Verification of the Quantum Fourier Transform (QFT) Using Qiskit

AIM:

To construct the Quantum Fourier Transform and its inverse for one, two and three qubits in Qiskit using Hadamard, controlled-phase and SWAP gates, and to verify the circuits against the QFT matrix.

APPARATUS REQUIRED:

- Python 3.10 or above
- Qiskit 2.x and Qiskit Aer simulator (`pip install qiskit qiskit-aer`)
- Matplotlib and pylatexenc (for circuit diagrams and histograms)
- NumPy (for matrix verification)
- Jupyter Notebook / Visual Studio Code (VS Code)
- Internet Connection (for package installation and documentation access)
- Quantum Computing Reference Material or Lab Manual

THEORY:

The Quantum Fourier Transform (QFT) is the quantum version of the discrete Fourier transform. It acts on the amplitudes of a quantum state. For n qubits there are $N = 2^n$ basis states, and the QFT maps the basis state $|j\rangle$ to a superposition in which every amplitude has the same size and a phase that depends on j:

$$\text{QFT}\,|j\rangle = \frac{1}{\sqrt{N}} \sum_{k=0}^{N-1} e^{2\pi i\, jk/N}\ |k\rangle$$

The entry in row k and column j of the QFT matrix is therefore $\omega^{jk}/\sqrt{N}$ with $\omega = e^{2\pi i/N}$. For one qubit ($N = 2$, $\omega = -1$) the QFT is the Hadamard gate. For two qubits ($N = 4$, $\omega = i$) it is

$$\mathrm{QFT}_4 = \frac{1}{2}\begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & i & -1 & -i \\ 1 & -1 & 1 & -1 \\ 1 & -i & -1 & i \end{bmatrix}$$

Circuit. The QFT is built from two kinds of gate: the Hadamard gate and the controlled-phase gate, which multiplies the state by $e^{i\theta}$ only when both of its qubits are 1. Each qubit receives one Hadamard gate followed by controlled-phase gates with the angles $\pi/2, \pi/4, \pi/8$, ... from the other qubits. SWAP gates at the end reverse the order of the qubits. An n-qubit QFT uses n Hadamard gates, $n(n-1)/2$ controlled-phase gates and $\lfloor n/2 \rfloor$ SWAP gates.

This is about n^2 gates, while the classical fast Fourier transform needs about $N\log N = n\ 2^n$ operations. The comparison has a limit: the transformed amplitudes cannot be read out, because a measurement returns only one basis state. The QFT is therefore useful inside other algorithms, not as a faster replacement for the FFT.

Two uses. First, the QFT turns a number into phases, and the inverse QFT turns phases back into a number; this is how quantum phase estimation reads a phase. Second, when the input state repeats with period r, the output of the QFT is concentrated on the multiples of N/r. For three qubits, a state with period 2 gives only the outcomes 0 and 4. Shor's algorithm uses this property to find a period.

In Qiskit, the controlled-phase gate is `qc.cp(theta, control, target)`, and a ready-made transform is available as `QFTGate` in `qiskit.circuit.library`. Qiskit treats qubit 0 as the least significant bit. Build the circuit so that its matrix matches the formula above, and check it with NumPy.

PROCEDURE:

1. Install Qiskit and the Aer simulator with `pip install qiskit qiskit-aer matplotlib pylatexenc`, and print the installed version with `qiskit._version_`.
2. Open Jupyter Notebook or Visual Studio Code.
3. Import `QuantumCircuit` and `transpile` from `qiskit`, `Statevector` and `Operator` from `qiskit.quantum_info`, `AerSimulator` from `qiskit_aer` and `plot_histogram` from `qiskit.visualization`. Import NumPy as well.
4. Create a circuit with n qubits. Starting from the most significant qubit, apply a Hadamard gate and then the controlled-phase gates `cp` with the angles $\pi/2$, $\pi/4$, ... controlled by the lower qubits.
5. Add SWAP gates to reverse the order of the qubits.
6. Draw the circuit with `qc.draw("mpl")` before running it.
7. Obtain the matrix with `Operator(qc).data` and compare it, using `np.allclose`,

with the matrix computed from the formula $e^{2\pi i jk/N}/\sqrt{N}$.

8. Build the inverse transform with `qc.inverse()` and confirm that the QFT followed by its inverse returns the input state.

9. Apply the circuit to the required input states, display the amplitudes and phases with `Statevector`, and run measured circuits on `AerSimulator` with 1024 shots.

10. Compare the result with the hand calculation. Remember that Qiskit prints qubit 0 as the rightmost bit. Record the code, the circuit diagram, the output and a one-line inference under each program.

PROGRAM:

Program 1: Show that the one-qubit QFT is the Hadamard gate by comparing the two operators.

OUTPUT:

```
import numpy as np
from qiskit import QuantumCircuit
from qiskit.quantum_info import Operator, Statevector
from qiskit.circuit.library import QFTGate

H = np.array([[1, 1], [1, -1]], dtype=complex) / np.sqrt(2)
QFT1 = np.array([[1, 1], [1, -1]], dtype=complex) / np.sqrt(2)

print("Hadamard matrix:")
print(H)
print("\nOne-qubit QFT matrix:")
print(QFT1)
print("\nAre they equal?", np.allclose(H, QFT1))

qc = QuantumCircuit(1)
qc.h(0)
print("\nHadamard circuit:")
print(qc.draw(output="text"))
```

```
Hadamard matrix:
[[ 0.70710678+0.j  0.70710678+0.j]
 [ 0.70710678+0.j -0.70710678+0.j]]

One-qubit QFT matrix:
[[ 0.70710678+0.j  0.70710678+0.j]
 [ 0.70710678+0.j -0.70710678+0.j]]

Are they equal? True

Hadamard circuit:
   ┌───┐
q: ┤ H ├
   └───┘
```

Program 2: Construct the two-qubit QFT from two Hadamard gates, one controlled-phase gate and one SWAP gate. Display its matrix and compare it with the matrix in the theory.

OUTPUT:

```
qc2 = QuantumCircuit(2)
qc2.h(1)
qc2.cp(np.pi / 2, 0, 1)
qc2.h(0)
qc2.swap(0, 1)

print("Two-qubit QFT circuit:")
print(qc2.draw(output="text"))

U_circuit = Operator(qc2).data
U_theory = Operator(QFTGate(2)).data
print("\nCircuit matrix:")
print(np.round(U_circuit, 3))
print("\nQiskit QFTGate matrix:")
print(np.round(U_theory, 3))
print("\nEquivalent?", np.allclose(U_circuit, U_theory))
```

```
Two-qubit QFT circuit:
q_0: ─────■──────┤ H ├─X─
          │P(π/2) └───┘ │
q_1: ┤ H ├─■─────────────X─

Circuit matrix:
[[ 0.5+0.j   0.5+0.j   0.5+0.j   0.5+0.j ]
 [ 0.5+0.j   0. +0.5j -0.5+0.j  -0. -0.5j]
 [ 0.5+0.j  -0.5+0.j   0.5+0.j  -0.5+0.j ]
 [ 0.5+0.j  -0. -0.5j -0.5+0.j   0. +0.5j]]

Qiskit QFTGate matrix:
[[ 0.5+0.j   0.5+0.j   0.5+0.j   0.5+0.j ]
 [ 0.5+0.j   0. +0.5j -0.5+0.j  -0. -0.5j]
 [ 0.5+0.j  -0.5+0.j   0.5-0.j  -0.5+0.j ]
 [ 0.5+0.j  -0. -0.5j -0.5+0.j   0. +0.5j]]

Equivalent? True
```

Program 3: Construct the three-qubit QFT. Draw the circuit and display its 8 × 8 matrix rounded to three decimal places.

OUTPUT:

```
qc3 = QuantumCircuit(3)
qc3.h(2)
qc3.cp(np.pi / 2, 1, 2)
qc3.cp(np.pi / 4, 0, 2)
qc3.h(1)
qc3.cp(np.pi / 2, 0, 1)
qc3.h(0)
qc3.swap(0, 2)

print("Three-qubit QFT circuit:")
print(qc3.draw(output="text"))

U3 = Operator(qc3).data
print("\n8 x 8 matrix rounded to three decimal places:")
print(np.round(U3, 3))
```

```
Three-qubit QFT circuit:
q_0: ──────────■──────────■──────┤ H ├─X─
               │          │P(π/2)     │
q_1: ──────■───┼──────┤ H ├─■─────────┼─
           │P(π/2)  │P(π/4)           │
q_2: ┤ H ├─■────────■─────────────────X─

8 x 8 matrix rounded to three decimal places:
[[ 0.354+0.j     0.354+0.j     0.354+0.j     0.354+0.j     0.354+0.j
   0.354+0.j     0.354+0.j     0.354+0.j   ]
 [ 0.354+0.j     0.25 +0.25j   0.   +0.354j -0.25 +0.25j  -0.354+0.j
  -0.25 -0.25j  -0.   -0.354j  0.25 -0.25j ]
 [ 0.354+0.j     0.   +0.354j -0.354+0.j    -0.   -0.354j  0.354+0.j
   0.   +0.354j -0.354+0.j    -0.   -0.354j]
 [ 0.354+0.j    -0.25 +0.25j  -0.   -0.354j  0.25 +0.25j  -0.354+0.j
   0.25 -0.25j   0.   +0.354j -0.25 -0.25j ]
 [ 0.354+0.j    -0.354+0.j     0.354+0.j    -0.354+0.j     0.354+0.j
  -0.354+0.j     0.354+0.j    -0.354+0.j   ]
 [ 0.354+0.j    -0.25 -0.25j   0.   +0.354j  0.25 -0.25j  -0.354+0.j
   0.25 +0.25j  -0.   -0.354j -0.25 +0.25j ]
 [ 0.354+0.j    -0.   -0.354j -0.354+0.j     0.   +0.354j  0.354+0.j
  -0.   -0.354j -0.354+0.j     0.   +0.354j]
 [ 0.354+0.j     0.25 -0.25j  -0.   -0.354j -0.25 -0.25j  -0.354+0.j
  -0.25 +0.25j   0.   +0.354j  0.25 +0.25j ]]
```

Program 4: Verify the three-qubit QFT circuit against the matrix computed from the formula with NumPy (`np.allclose`), and against Qiskit's `QFTGate`.

OUTPUT:

```
def qft_matrix(n):
    N = 2 ** n
    x = np.arange(N)
    y = x.reshape((N, 1))
    return np.exp(2j * np.pi * y * x / N) / np.sqrt(N)

U_formula = qft_matrix(3)
U_qiskit = Operator(QFTGate(3)).data
U_manual = Operator(qc3).data

print("Manual circuit matches formula:", np.allclose(U_manual, U_formula))
print("Manual circuit matches QFTGate:", np.allclose(U_manual, U_qiskit))
print("\nFormula matrix:")
print(np.round(U_formula, 3))
```

```
Manual circuit matches formula: True
Manual circuit matches QFTGate: True

Formula matrix:
[[ 0.354+0.j     0.354+0.j     0.354+0.j     0.354+0.j     0.354+0.j
   0.354+0.j     0.354+0.j     0.354+0.j   ]
 [ 0.354+0.j     0.25 +0.25j   0.   +0.354j -0.25 +0.25j  -0.354+0.j
  -0.25 -0.25j  -0.   -0.354j  0.25 -0.25j ]
 [ 0.354+0.j     0.   +0.354j -0.354+0.j    -0.   -0.354j  0.354-0.j
   0.   +0.354j -0.354+0.j    -0.   -0.354j]
 [ 0.354+0.j    -0.25 +0.25j  -0.   -0.354j  0.25 +0.25j  -0.354+0.j
   0.25 -0.25j   0.   +0.354j -0.25 -0.25j ]
 [ 0.354+0.j    -0.354+0.j     0.354-0.j    -0.354+0.j     0.354-0.j
  -0.354+0.j     0.354-0.j    -0.354+0.j   ]
 [ 0.354+0.j    -0.25 -0.25j   0.   +0.354j  0.25 -0.25j  -0.354+0.j
   0.25 +0.25j  -0.   -0.354j -0.25 +0.25j ]
 [ 0.354+0.j    -0.   -0.354j -0.354+0.j     0.   +0.354j  0.354-0.j
  -0.   -0.354j -0.354+0.j    -0.   +0.354j]
 [ 0.354+0.j     0.25 -0.25j  -0.   -0.354j -0.25 -0.25j  -0.354+0.j
  -0.25 +0.25j  -0.   +0.354j  0.25 +0.25j ]]
```

Program 5: Apply the three-qubit QFT to $|000\rangle$ and to $|101\rangle$. Display the amplitudes, their phases and the probabilities, and explain why all eight probabilities are equal.

OUTPUT:

```
def show_qft_output(bitstring):
    n = len(bitstring)
    index = int(bitstring, 2)
    initial = Statevector.from_int(index, 2 ** n)
    output = initial.evolve(Operator(QFTGate(n)))

    print(f"Input |{bitstring}>")
    print("Basis |y>   Amplitude                Phase (radians)   Probability")
    for y, amp in enumerate(output.data):
        print(f"|{y:03b}>      {amp.real:+.3f}{amp.imag:+.3f}j       "
              f"{np.angle(amp):+.3f}          {abs(amp)**2:.3f}")
    print("Sum of probabilities:", np.sum(np.abs(output.data) ** 2).real)
    print()

show_qft_output("000")
show_qft_output("101")
```

```
Input |000>
Basis |y>   Amplitude                Phase (radians)   Probability
|000>      +0.354+0.000j       +0.000          0.125
|001>      +0.354+0.000j       +0.000          0.125
|010>      +0.354+0.000j       +0.000          0.125
|011>      +0.354+0.000j       +0.000          0.125
|100>      +0.354+0.000j       +0.000          0.125
|101>      +0.354+0.000j       +0.000          0.125
|110>      +0.354+0.000j       +0.000          0.125
|111>      +0.354+0.000j       +0.000          0.125
Sum of probabilities: 1.0000000000000002

Input |101>
Basis |y>   Amplitude                Phase (radians)   Probability
|000>      +0.354+0.000j       +0.000          0.125
|001>      -0.250-0.250j       -2.356          0.125
|010>      +0.000+0.354j       +1.571          0.125
|011>      +0.250-0.250j       -0.785          0.125
|100>      -0.354+0.000j       +3.142          0.125
|101>      +0.250+0.250j       +0.785          0.125
|110>      -0.000-0.354j       -1.571          0.125
|111>      -0.250+0.250j       +2.356          0.125
Sum of probabilities: 1.0
```

Program 6: Construct the inverse QFT and verify that the QFT followed by the inverse QFT returns the input for all eight basis states.

OUTPUT:

```
qft_gate = QFTGate(3)
iqft_gate = qft_gate.inverse()

for x in range(8):
    state = Statevector.from_int(x, 8)
    result = state.evolve(Operator(qft_gate)).evolve(Operator(iqft_gate))
    expected = Statevector.from_int(x, 8)
    print(f"|{x:03b}> returned correctly:", result.equiv(expected))

iqft_circuit = QuantumCircuit(3)
iqft_circuit.append(iqft_gate, [0, 1, 2])
print("\nInverse QFT circuit:")
print(iqft_circuit.draw(output="text"))
```

```
|000> returned correctly: True
|001> returned correctly: True
|010> returned correctly: True
|011> returned correctly: True
|100> returned correctly: True
|101> returned correctly: True
|110> returned correctly: True
|111> returned correctly: True

Inverse QFT circuit:
     ┌─────────┐
q_0: ┤0        ├
     │         │
q_1: ┤1 qft_dg ├
     │         │
q_2: ┤2        ├
     └─────────┘
```

Program 7: Write a function `qft(n)` that returns the QFT circuit for any number of qubits. Build it for n = 4, report the number of each gate and the circuit depth, and compare the gate counts with the formulas in the theory.

OUTPUT:

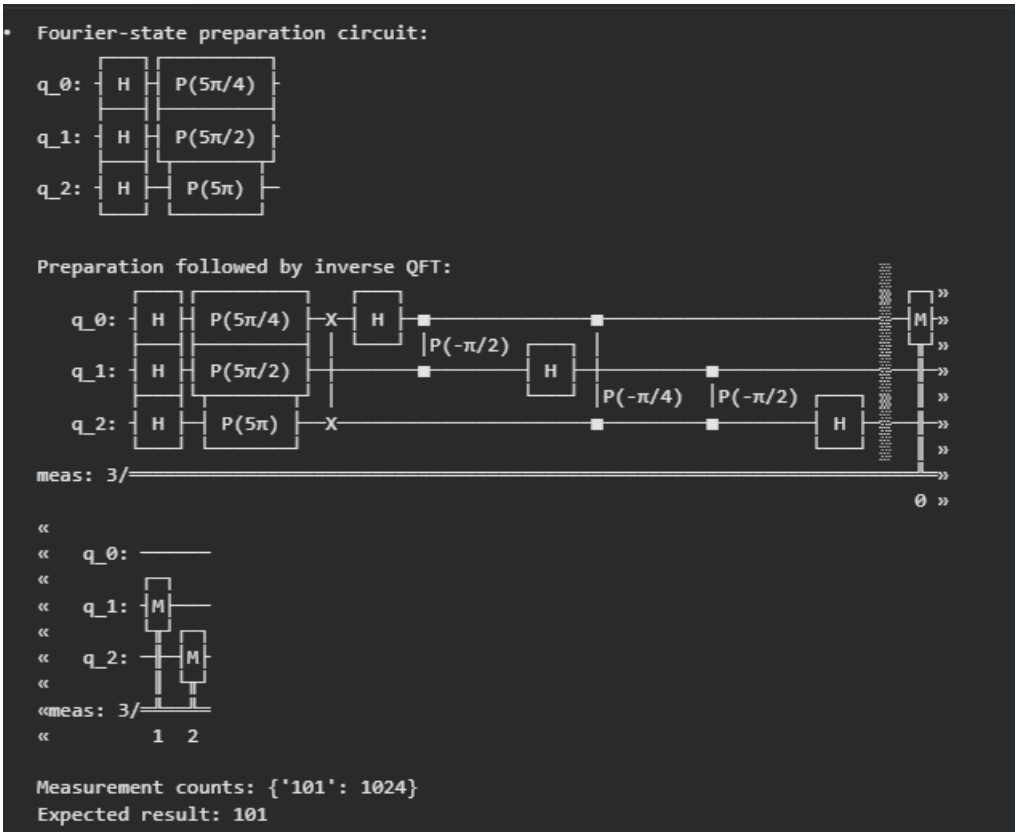
```
from collections import Counter

def qft(n, do_swaps=True):
    circuit = QuantumCircuit(n, name=f"QFT{n}")
    for j in range(n - 1, -1, -1):
        circuit.h(j)
        for k in range(j - 1, -1, -1):
            circuit.cp(np.pi / (2 ** (j - k)), k, j)
    if do_swaps:
        for i in range(n // 2):
            circuit.swap(i, n - i - 1)
    return circuit

qc4 = qft(4)
print("Four-qubit QFT circuit:")
print(qc4.draw(output="text"))

counts = Counter(inst.operation.name for inst in qc4.data)
print("\nGate counts:", dict(counts))
print("Circuit depth:", qc4.depth())
print("Expected H gates:", 4)
print("Expected controlled-phase gates:", 4 * (4 - 1) // 2)
print("Expected SWAP gates:", 4 // 2)
print("Expected total gates:", 4 + 4 * (4 - 1) // 2 + 4 // 2)
```

```
Four-qubit QFT circuit:
q_0: ──────────────
```



Program 9: Prepare the three-qubit state with period 2 (an equal superposition of 0, 2, 4 and 6), apply the QFT and measure. Show that only the outcomes 0 and 4 appear. Repeat for a state with period 4 and explain the pattern.

OUTPUT:

```python
def run_periodic_state(period, n=3):
    N = 2 ** n
    values = list(range(0, N, period))
    vector = np.zeros(N, dtype=complex)
    vector[values] = 1 / np.sqrt(len(values))

    initial = Statevector(vector)
    output = initial.evolve(Operator(QFTGate(n)))
    probabilities = np.abs(output.data) ** 2

    print(f"\nPeriod {period}")
    print("Input basis values:", values)
    print("Output probabilities:", np.round(probabilities, 3))
    print("Significant output indices:",
          [i for i, p in enumerate(probabilities) if p > 1e-8])
    return probabilities

period2_probs = run_periodic_state(2)
period4_probs = run_periodic_state(4)
```

```
Period 2
Input basis values: [0, 2, 4, 6]
Output probabilities: [0.5 0.  0.  0.  0.5 0.  0.  0. ]
Significant output indices: [0, 4]

Period 4
Input basis values: [0, 4]
Output probabilities: [0.25 0.   0.25 0.   0.25 0.   0.25 0.  ]
Significant output indices: [0, 2, 4, 6]
```

VIVA Q&A:

1. What does the Quantum Fourier Transform do to a basis state?
It maps |x⟩ to an equal superposition of all basis states, with a phase that depends on x and the output index.

2. How is the QFT related to the classical discrete Fourier transform?
It is the quantum operation corresponding to the discrete Fourier transform on amplitudes. It transforms amplitudes rather than directly outputting all classical Fourier coefficients.

3. Why is the one-qubit QFT the same as the Hadamard gate?
For one qubit, the QFT matrix is [[1, 1], [1, −1]] divided by √2, which is exactly the Hadamard matrix.

4. Which gates are used to build an n-qubit QFT, and how many of each?
The standard circuit uses n Hadamard gates, n(n−1)/2 controlled-phase gates, and floor(n/2) SWAP gates if the final bit-reversal swaps are included.

5. What does the controlled-phase gate do, and which angles appear in the QFT?
It adds a phase to the |11⟩ component. The QFT uses angles π/2, π/4, π/8, and so on, according to the separation between qubits.

6. Why are SWAP gates placed at the end of the QFT circuit?
They reverse the qubit order so that the output matches the standard binary ordering.

7. Why can the QFT not be used as a faster FFT to read out all the Fourier coefficients?
Measurement gives only a limited classical sample of the quantum state. It does not reveal all amplitudes in one run.

8. What is the inverse QFT, and where is it used?
It is the inverse operation of the QFT. It is used in quantum phase estimation, Shor's algorithm, and decoding Fourier states.

9. What does the QFT output look like for an input state with period r?
It produces peaks at regularly spaced frequency values. The exact spacing depends on the register size and the period.

10. How does the number of gates in the QFT compare with the number of operations in the FFT?
The standard QFT circuit uses $O(n^2)$ gates for n qubits, while the FFT uses O(N log N) classical operations for N data points. Since $N=2^n$, these complexities refer to different input/output models and should not be compared as if they perform the same task.

RESULT:

The QFT was constructed for one, two, three, and four qubits; its matrix and inverse were verified; basis-state and periodic-state behavior were examined; and the gate-count formulas were compared with the circuit.

EVALUATION (for faculty use):

Evaluation method	Prep	Program	Output	Inference	Viva	Timely	Total
Maximum marks	1	4	2	1	1	1	10
Marks awarded							

Faculty signature with date: